

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

PANDAS: DataFrames (cont.)

DataFrame: filtros

Como vimos en la clase pasada, una forma de acceder a los elementos de un DataFrame es utilizando el nombre de una columna. El resultado es un objeto Series que contiene los valores de dicha columna.

```
print(df)
''' Imprime:
   nombre  edad sexo  peso  altura  colesterol
0  Martín González   18   H   85    1.79         182
1  Lucía Fernández   32   M   65    1.73         232
2    Sofía López    35   M   65    1.70         200
3   María Gómez    46   M   51    1.58         148
4    Juan Ruiz     68   H   66    1.74         249
'''

print(df['colesterol'])
''' Imprime:
0    182
1    232
2    200
3    148
4    249
Name: colesterol, dtype: float64
'''
```



DataFrame: filtros

En pandas, para filtrar filas se coloca una condición dentro de `[ ]`. La condición devuelve valores **True/False** que indican qué filas se incluyen en el nuevo DataFrame.

```
print(df)
''' Imprime:
   nombre  edad sexo  peso  altura  colesterol
0  Martín González   18   H   85    1.79         182
1  Lucía Fernández   32   M   65    1.73         232
2    Sofía López    35   M   65    1.70         200
3  María Gómez    46   M   51    1.58         148
4    Juan Ruiz    68   H   66    1.74         249  '''

print(df['colesterol']>230)
''' Imprime:
0    False
1     True
2    False
3    False
4     True
Name: colesterol, dtype: bool  '''

print(df[df['colesterol']>230])
''' Imprime:
   nombre  edad sexo  peso  altura  colesterol
1  Lucía Fernández   32   M   65    1.73         232
4    Juan Ruiz    68   H   66    1.74         249  '''
```

DataFrame: filtros

Al filtrar un DataFrame se pueden usar condiciones compuestas. Los operadores booleanos son: ~ (negación), & (AND) y | (OR).

```
print(df)
''' Imprime:
      nombre  edad sexo  peso  altura  colesterol
0  Martín González   18   H   85   1.79         182
1  Lucía Fernández   32   M   65   1.73         232
2    Sofía López     35   M   65   1.70         200
3   María Gómez     46   M   51   1.58         148
4     Juan Ruiz     68   H   66   1.74         249  '''

print(df[(df['sexo'] == 'H') & (df['colesterol']>200)])
''' Imprime:
      nombre  edad sexo  peso  altura  colesterol
4   Juan Ruiz     68   H   66   1.74         249  '''

print(df[(df['sexo'] == 'H') | (df['colesterol']>200)])
''' Imprime:
      nombre  edad sexo  peso  altura  colesterol
0  Martín González   18   H   85   1.79         182
1  Lucía Fernández   32   M   65   1.73         232
4     Juan Ruiz     68   H   66   1.74         249  '''
```

Ejercicio: filtros

Ejercicio 1. Se cuenta con un archivo `estudiantes.csv` que contiene información sobre alumnos:

```
nombre,edad,carrera,promedio,correo
Norberto Catalá Adán,20,Economía,7.69,oangulo@hotmail.com
Leandro Lledó Escalona,29,Derecho,8.27,ghuerta@yahoo.com
Maxi Riquelme Guillen,18,Ingeniería,8.3,rosalindaabad@yahoo.com
Jesusa Figuerola Palomares,21,Medicina,8.93,nsalvador@hotmail.com
Agustín Polo,22,Medicina,6.51,piglesias@gmail.com
...
```

Se pide:

1. Crear un `DataFrame` `df` a partir del archivo `estudiantes.csv`.
2. Mostrar las primeras 10 filas de `df`.
3. Seleccionar los estudiantes de la carrera Medicina.
4. Seleccionar los estudiantes con promedio mayor a 9.
5. Seleccionar los estudiantes de Ingeniería con promedio menor a 7.
6. Seleccionar los estudiantes de Derecho o Economía con promedio mayor a 9.
7. Seleccionar los estudiantes que *no* sean de Medicina.
8. Seleccionar los estudiantes de Ingeniería con promedio mayor a 9 y mostrar únicamente sus columnas `nombre` y `promedio`.

DataFrame: agrupamientos

Muchas veces es útil agrupar los datos de un DataFrame por una o varias columnas para facilitar determinados análisis.

	Nombre	Edad	Sexo	Colesterol
0	Ana	45	Mujer	210
1	Luis	52	Hombre	195
2	Marta	37	Mujer	230
3	Pedro	49	Hombre	185
4	Carla	41	Mujer	205



Agrupación por Hombres

	Sexo	Colesterol Prom.
0	Hombre	190.0



Agrupación por Mujeres

	Sexo	Colesterol Prom.
0	Mujer	215.0

DataFrame: agrupamientos

En pandas, esto se logra con `groupby(...)` junto con funciones de agregación.

```
print(df)
''' Imprime:
  Nombre  Edad  Sexo  Colesterol
0    Ana   45   Mujer         210
1   Luis   52  Hombre         195
2   Marta  37   Mujer         230
3   Pedro  49  Hombre         185
4   Carla  41   Mujer         205
'''

# Agrupamos por Sexo y calculamos promedio de Colesterol
resultado = df.groupby("Sexo")["Colesterol"].mean()
print(resultado)

''' Imprime:
Sexo
Hombre    190.0
Mujer     215.0
Name: Colesterol, dtype: float64
'''
```

El resultado es **un nuevo objeto de tipo Series**.

DataFrame: agrupamientos

Los agrupamientos pueden realizarse utilizando más de una columna.

```
print(df)
''' Imprime:
   Nombre  Edad   Sexo  Colesterol
0     Ana   45   Mujer         210
1     Luis  52   Hombre         195
2     Marta 37   Mujer         230
3     Pedro 49   Hombre         185
4     Carla 41   Mujer         205
5     Juan  49   Hombre         200
6     Sofía 37   Mujer         240  '''

# Agrupamos por Sexo y Edad y calculamos promedio de Colesterol
resultado = df.groupby(["Sexo", "Edad"])["Colesterol"].mean()
print(resultado)

''' Imprime:
   Sexo  Edad
Hombre   49  192.5
         52  195.0
Mujer    37  235.0
         41  205.0
         45  210.0
Name: Colesterol, dtype: float64  '''
```

- ▶ Dos hombres de 49 años (promedio 192.5)
- ▶ Dos mujeres de 37 años (promedio 235)

DataFrame: agrupamientos

Las principales funciones de agregación disponibles en pandas son:

- ▶ `mean()` → Promedio de los valores del grupo
- ▶ `sum()` → Suma de los valores del grupo
- ▶ `size()` → Cantidad total de elementos
- ▶ `min()` → Valor mínimo del grupo
- ▶ `max()` → Valor máximo del grupo
- ▶ `median()` → Mediana de los valores del grupo
- ▶ `std()` → Desviación estándar del grupo
- ▶ `var()` → Varianza del grupo
- ▶ `first()` → Primer valor del grupo
- ▶ `last()` → Último valor del grupo
- ▶ `nunique()` → Cantidad de valores únicos del grupo

DataFrame: agrupamientos

A un agrupamiento se le pueden aplicar múltiples funciones de agregación.

```
print(df)
''' Imprime:
   Nombre  Edad  Sexo  Colesterol
0     Ana   45  Mujer         210
1     Luis   52  Hombre         195
2     Marta  37  Mujer         230
3     Pedro  49  Hombre         185
4     Carla  41  Mujer         205
5     Juan   49  Hombre         200
6     Sofía  37  Mujer         240  '''

# Agrupamos por Sexo y aplicamos múltiples funciones de agregación
resultado = df.groupby("Sexo")["Colesterol"].agg(["mean", "min", "max"])
print(resultado)

''' Imprime:
      mean min max
Sexo
Hombre   193.3 185 200
Mujer    221.2 205 240
'''
```

Cuando se aplican múltiples funciones de agregación, **el resultado es un objeto de tipo DataFrame.**

Ejercicio: agrupamientos

Ejercicio 2. Se cuenta con un archivo CSV `estudiantes.csv` que contiene información sobre alumnos:

```
nombre,edad,carrera,promedio,correo
Norberto Catalá Adán,20,Economía,7.69,oangulo@hotmail.com
Leandro Lledó Escalona,29,Derecho,8.27,ghuerta@yahoo.com
Maxi Riquelme Guillen,18,Ingeniería,8.3,rosalindaabad@yahoo.com
Jesusa Figuerola Palomares,21,Medicina,8.93,nsalvador@hotmail.com
Agustín Polo,22,Medicina,6.51,piglesias@gmail.com
...
```

Se pide:

1. Crear un DataFrame `df` a partir del archivo `estudiantes.csv`.
2. Agrupar los datos por carrera y calcular el promedio de promedio.
3. Agrupar los datos por carrera y obtener las funciones de agregación `mean`, `min` y `max` sobre la columna `promedio`.
4. Contar cuántos estudiantes hay en cada carrera.
5. Identificar qué carrera tiene el promedio más alto y cuál el más bajo.

Repaso de la clase de hoy

Referencia: [Documentación de pandas \(link\)](#)

DataFrame

- ▶ Filtros
- ▶ Funciones de agregación

Con lo visto, ya pueden completar la sección 1 de la guía de Métodos Analíticos.